

- ❖ Define token, terminal, nonterminal, and syntax-directed translation. Write the four components of context free grammar. What is recursive-decent parsing? How do you eliminate the left-recursive from a grammar?

- A **token** is the smallest unit of a program that has meaning to the compiler or interpreter.
- **Terminals** are the "building blocks" of a language.
 $\text{stmt} \rightarrow \text{if}(\text{expr}) \text{stmt} \text{ else } \text{stmt}$
 Lexical elements like keyword *if*, *parentheses*.
- **Non-terminals** help define how those blocks are structured. Variables like *expr*, *stmt*.
- A **grammar-oriented** compiling technique known as syntax-directed translation.

A context-free grammar has four components:

1. A set of terminals
2. A set of non-terminals
3. A set of production
4. A designation of one of the non-terminals as the *start* symbol.

Recursive-decent is a method where each grammar rule is implemented as a recursive function to parse the input.

Example: For a rule like $E \rightarrow T + E$, the parser has a function for E that calls itself for T and E.

Left Recursion and Removal

Left recursion happens when a rule calls itself at the start, like:

$$A \rightarrow A\alpha \mid \beta$$

This causes infinite loops in recursive-descent parsing.

Fix: Rewrite it to remove left recursion:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

- ❖ What is the function of the lexical analyzer? How does the lexical analyzer recognize keywords and identifiers in a source program? Who creates symbol-table entries?

Functions of Lexical Analyzer

1. Convert the source code into stream of tokens
2. Removing white spaces
3. Removing comments
4. Recognizing keyword & Identifier
5. Recognizing of constants

How the Lexical Analyzer Recognizes Keywords and Identifiers

- **Tokenization:** The lexical analyzer scans the source program and breaks it into tokens (smallest meaningful units).
- **Keyword Recognition:** Keywords (like if, while) are stored in a reserved keyword list. The lexical analyzer checks each token against this list.
- **Identifier Recognition:** Tokens not in the keyword list but following naming rules (like variable names) are recognized as identifiers.

Who Creates Symbol Table Entries?

- The **lexical analyzer** creates entries for identifiers in the symbol table.
- It adds details like the identifier's name, type, and scope when first recognized.

- ❖ Define tokens, patterns, and lexemes with examples. What is the function of the lexical analyzer? Demonstrate the Interactions between the lexical analyzer and the parser.

❖ **What is the role of parser? Define left-recursive and non-left-recursive grammars with examples.**

Role of the Parser

The parser processes tokens from the lexical analyzer to:

- Analyze syntactic structure of source code.
 - Construct parse tree
 - Check syntactic correctness, detects errors, generate meaningful order.
 - Output serves as input for subsequent compiler phases.
- A grammar is **left-recursive** if a rule starts with itself. This can cause problems like infinite loops in parsers.

$$A \rightarrow A\alpha \mid \beta$$

Here, A starts with itself in the rule $A \rightarrow A\alpha$.

- A grammar is **non-left-recursive** if a rule does not start with itself. These grammars work better for parsers.

$$\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$$

$$\text{Expr} \rightarrow \text{Term Expr}'$$

$$\text{Expr}' \rightarrow + \text{Term Expr}' \mid \epsilon$$

❖ **Explain the formal definition of a context-free grammar. How to eliminate the left recursion from the production.**

A **context-free grammar (CFG)** is a system of rules used to describe the structure of a language.

Formal notation:

A CFG is written as:

$$G = (V, \Sigma, S, P)$$

Where:

- V = Set of non-terminals.
- Σ = Set of terminals.
- S = Start symbol ($S \in V$).
- P = Set of production rules (e.g., $S \rightarrow aS \mid \epsilon$).

❖ **Define syntax-directed translation. What are the four components of context-free grammar? How to eliminate the left-recursive production?**

❖ **Mention the property of ambiguous grammar. Write the scanning process of top-down parsing on input. What is the function of a lexical analyzer? Write a pseudo code for skipping white space.**

A grammar can have more than one parse tree generating a given string of terminals. Such a grammar is said to be ambiguous.

Repeatedly performing the following two steps.

1. At node N, labeled with nonterminal A, select one of the productions for A and construct children at N for the symbols in the production body.
2. Find the next node at which a subtree is to be constructed, typically the leftmost unexpanded nonterminal of the tree. For some grammars, the above steps can be implemented during a single left-to-right scan of the input string.

Scanning process of top-down parsing on input:

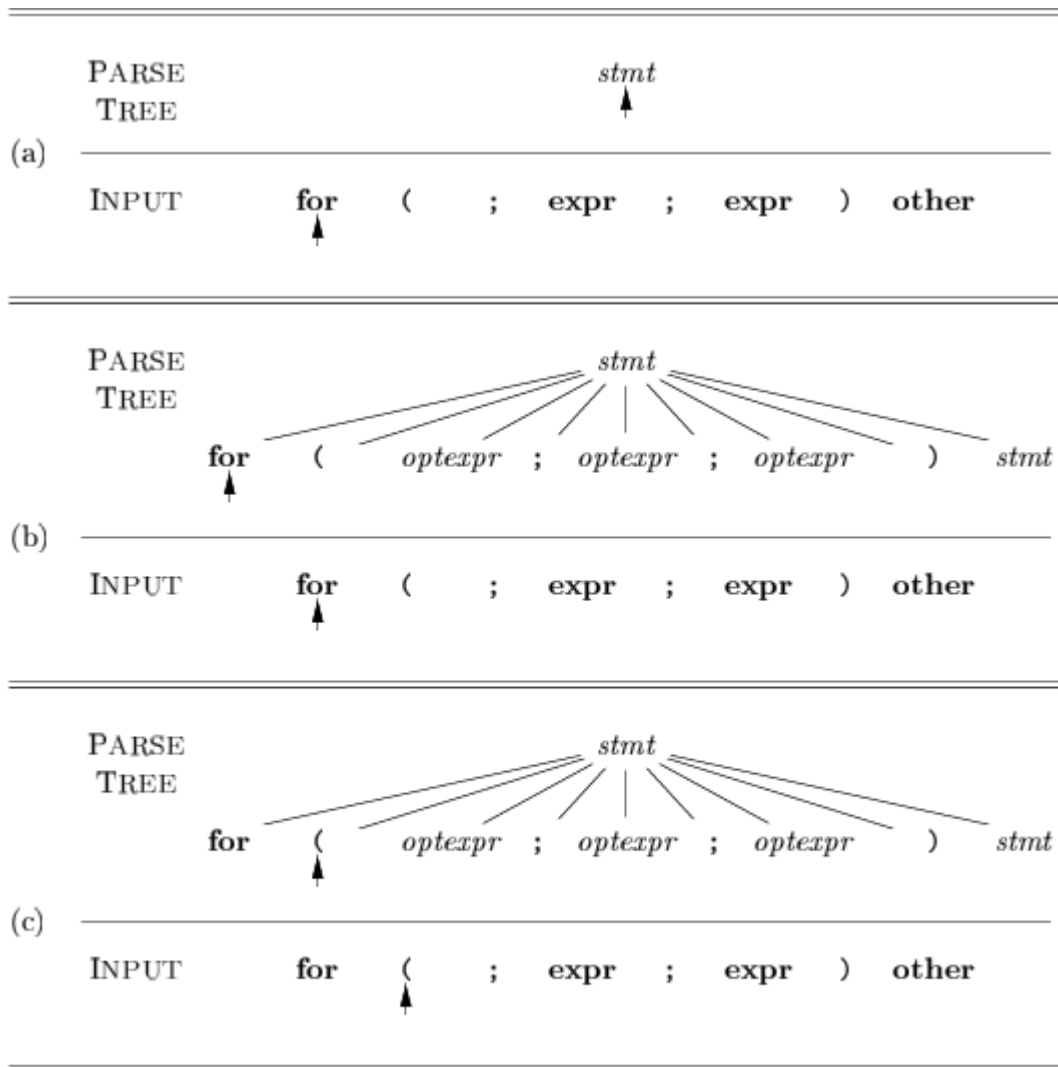


Figure 2.18: Top-down parsing while scanning the input from left to right

Lexical analyzer separates the characters of the source language into groups that logically belong together, called tokens.

```
for ( ; ; peek = next input character ) {  
    if ( peek is a blank or a tab ) do nothing;  
    else if ( peek is a newline ) line = line+1;  
    else break;  
}
```

- ❖ Write the main task of the lexical analyzer. Define tokens, patterns, and lexemes. Explore the tokens, patterns, and lexemes from the C statement: `printf("Total = %d\n", scores);`

- The main task of lexical analyzer is to read the input characters of the source program and group them into lexemes
- A **token** is the smallest unit of a program that has meaning to the compiler or interpreter.
- **Lexeme** is a sequence of character in the source code that matches the pattern for a particular token.
- **Pattern** is a rule or specification that describe the set of possible lexemes for a particular tokens.

Token	Pattern	Lexeme
Identifier	Alphanumeric or underscore	printf, scores
Special Symbol	Punctuation characters (e.g., (,))	(,), ,, ;
String Literal	Text enclosed in double quotes	"Total = %d\n"

- ❖ What is the role of the parser? Illustrate the position of the parser in the compiler model. Define left-recursive and non-left-recursive grammar with examples.

Role of the Parser

The parser processes tokens from the lexical analyzer to:

- Analyze syntactic structure of source code.
- Construct parse tree
- Check syntactic correctness, detects errors, generate meaningful order.
- Output serves as input for subsequent compiler phases.

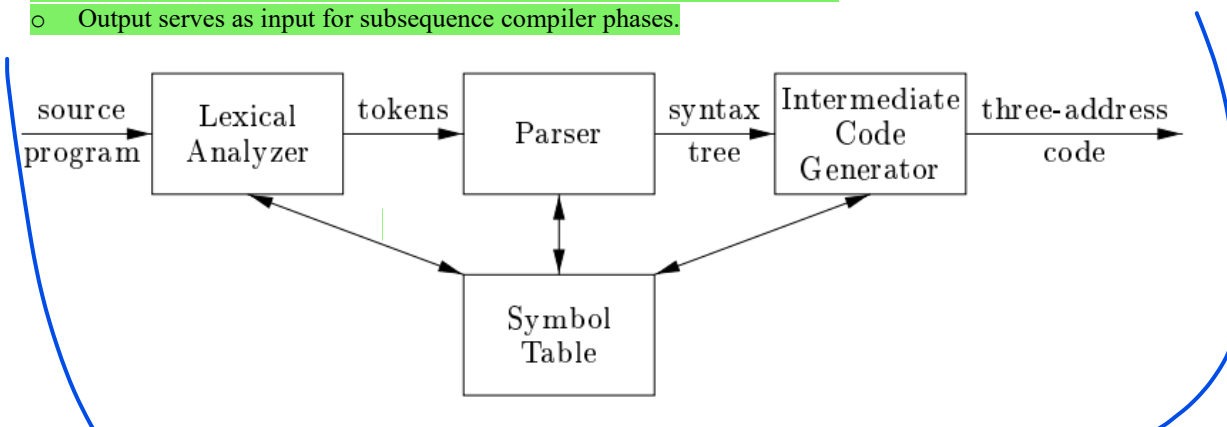


Figure 2.3: A model of a compiler front end

- ❖ What are the benefits of having lexical rules out of the grammar of a parser? Explain ambiguity with proper example.

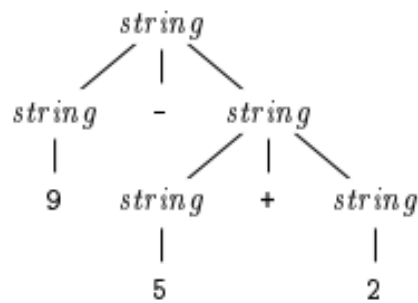
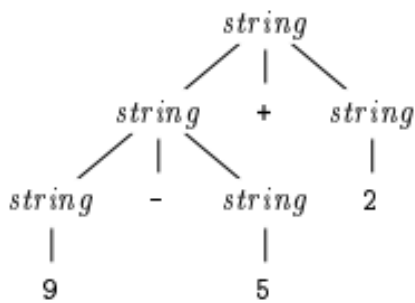
Separating lexical rules from the parser:

- Simplifies Grammar – Keeps syntax rules clean.
- Faster Parsing – Lexing is quicker than parsing.
- Better Error Handling – Detects invalid tokens early.
- Modular Design – Lexer and parser work independently.
- Optimized Processing – Skips whitespace, handles comments.

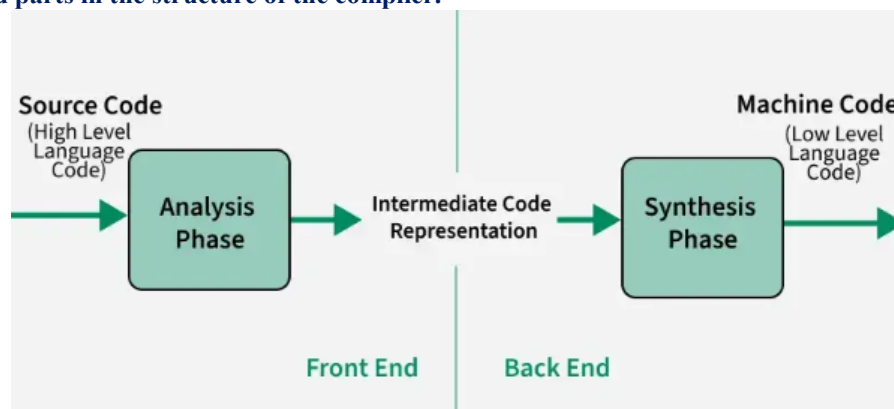
This figure shows ambiguity in parsing, where the expression $9 - 5 + 2$ has two different parse trees:

- **Left Tree (Left Associative):** $9 - 5$ is calculated first, then $4 + 2 = 6$.
- **Right Tree (Right Associative):** $5 + 2$ is calculated first, then $9 - 7 = 2$.

The ambiguity occurs because the grammar doesn't define precedence or associativity. Fixing these rules removes the ambiguity.

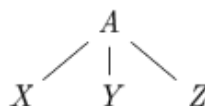


- ❖ **Illustrate the typical decomposition of a compiler into phases (already noted). Identify the front-end and the back-end parts in the structure of the compiler.**



- ❖ **What is Parse Tree? Write the properties of a Parse Tree according to consider a context-free grammar.**

A parse tree pictorially shows how the start symbol of a grammar derives a string in the language. If non-terminal A has a production $A \rightarrow XYZ$, then a parse tree may have an interior node labeled A with three children labeled X , Y , and Z , from left to right:



Formally, given a context-free grammar, a parse tree according to the grammar is a tree with the following properties:

1. The root is labeled by the start symbol.
2. Each leaf is labeled by a terminal or by ϵ .
3. Each interior node is labeled by a nonterminal.
4. If A is the nonterminal labeling some interior node and $X_1, X_2, \dots, X_n$ are the labels of the children of that node from left to right, then there must be a production $A \rightarrow X_1 X_2 \dots X_n$. Here, $X_1, X_2, \dots, X_n$ each stand for a symbol that is either a terminal or a nonterminal. As a special case, if $A \rightarrow \epsilon$ epsilon is a production, then a node labeled A may have a single child labeled

❖ Difference Between Inherited and Synthesized Attributes

Aspect	Inherited Attributes	Synthesized Attributes
Direction	Top-down (from parent to child) or between siblings .	Bottom-up (from child to parent).
Defined By	Parent or sibling nodes.	Child nodes.
Usage	Used for contextual dependencies (e.g., type checking, variable scope).	Used for computed properties (e.g., expression evaluation).
Example	$E' \rightarrow + T E'$ (Passes inherited attribute from E to E').	$E \rightarrow T E'$ ($E.value = T.value + E'.value$).